
NEORV32 Accelerator for Cryptography

Ascon-AEAD128 Hardware Accelerator
Development Report

Author	Mohammadi Masoudi Ramin
Project Title	NEORV32 Accelerator for Cryptography
Companion	RASD, Rev. 0.5 (May 15, 2026)
Target Platform	NEORV32 RISC-V SoC, Sipeed Tang Nano 9K
FPGA Device	Gowin GW1NR-9 (8640 LUT4, 6480 FF, 26 BSRAM)
Integration	NEORV32 Custom Functions Subsystem (CFS)
Algorithm	Ascon-AEAD128, NIST SP 800-232 (Aug. 2025)
HDL	Verilog accelerator; VHDL CFS wrapper
Document Rev.	1.0
Date	May 20, 2026

This document is the development companion to the Requirements & Design Document (RASD). It records every implementation decision, the four-layer verification strategy and its concrete evidence, the synthesis and silicon-validation results, the configurable architecture-exploration framework, and a side-by-side comparison with the official Ascon hardware reference (Primas et al., IAIK Graz). The RASD specifies what the system shall do; this document records what was built, what was learned, and what was deliberately chosen along the way.

Contents

1	Introduction	2
1.1	Purpose	2
1.2	Relationship to the RASD	2
1.3	Document Conventions	2
2	System Overview	3
2.1	High-Level Architecture	3
2.2	Integrated System Block Diagram	4
2.3	Verification Pyramid	4
3	Design Decisions	4
3.1	Foundational Decisions	5
3.2	RTL Decisions	6
3.3	Integration Decisions	7
3.4	Firmware Decisions	8
3.5	Verification Decisions	8
3.6	Process and Repository Decisions	9
4	Configuration Space: 560 Architecturally Valid Points	9
4.1	The Four Architectural Axes	9
4.2	Visualising the Design Space	10
4.3	What Was Implemented	10
5	Implementation Status, by RASD Requirement	10
5.1	Functional Requirements	11
5.2	Non-Functional Requirements	12
6	Verification Methodology	13
6.1	Layer 1 — Python golden model	13
6.2	Layer 2 — Verilog RTL against the model	13
6.3	Layer 3 — C driver vs. emulated transport	13
6.4	Layer 4 — End-to-end co-simulation	13
6.4.1	Result	14
7	Synthesis and Silicon Validation	15
7.1	Standalone Backends on the Tang Nano 9K	15
7.2	Integrated NEORV32 + ASCON-CFS	15
8	Architecture Exploration Framework	16
8.1	Five FPGA Backend Variants	16
8.2	The AXI-MMIO Bridge	16
8.3	What is <i>not yet</i> implemented	16
9	Hash, XOF, and CXOF128 Backend	17
9.1	Implementation	17
9.2	Verification	17
9.3	Integration status	18
10	Comparison: this project vs. <code>ascon/ascon-hardware</code>	18
10.1	Scope and Design Philosophy	18
10.2	Interface, Integration, and Verification	19
10.3	Per-Payload Sweep on the Integrated SoC (Co-Simulation)	20
10.4	Implementation and Resource Comparison	20
10.5	Author Note on the Performance Comparison Section	21
11	Toolchain and Bring-Up Findings	21
11.1	NEORV32 <code>BOOT_MODE_SELECT</code>	21
11.2	Soft-float vs. double-float multilib	22
11.3	NEORV32 <code>printf</code> width-modifier limitation	22
11.4	Makefile tab-discipline in the <code>cosim</code> flow	22
12	Remaining Work	22
13	Conclusions	22
A	Tools and Versions	23
B	Reproducing Every Result	23
C	Glossary	24
D	Document History	24

1 Introduction

1.1 Purpose

This document records the development of an Ascon-AEAD128 hardware accelerator integrated as a Custom Functions Subsystem (CFS) on the NEORV32 RISC-V soft-core, together with the broader architecture-exploration framework that grew up around the single-instance accelerator. It is the technical record that the RASD §10.7 reserved to a “separate development report”.

1.2 Relationship to the RASD

RASD (Rev. 0.5)	Development Report (this document)
Defines the algorithm, the register map, the FSM, and the functional and non-functional requirements that the implementation <i>shall</i> satisfy.	Records what was actually built, the design decisions that were taken, the verification evidence collected, the synthesis numbers measured, the architecture-exploration framework, and the comparison against the official ascon-hardware reference.
Scope: Ascon-AEAD128 on NEORV32 + Tang Nano 9K.	Scope: everything in the RASD plus the broader <i>generator</i> that was developed during the work — five backend variants, an AXI-stream data plane, an AXI-MMIO bridge, the configuration vocabulary in <code>ascon_arch/</code> , and the multi-layer verification flow that includes a Verilator co-simulation of the integrated SoC.
References future development.	Lists every design decision (§3), maps every RASD requirement to its implementation (§5), and shows what was verified and where (§6).

1.3 Document Conventions

Box type	Meaning
FR-xxx	RASD functional requirement (referenced; the full text is in the RASD).
NFR-xxx	RASD non-functional requirement (referenced).
DD-xxx	A design decision made during development. Each lists rationale, alternatives considered, consequences, and where it is implemented.
Important	A pitfall or non-obvious constraint future readers should know about.

2 System Overview

2.1 High-Level Architecture

Figure 1 shows the system organised as five horizontal layers, each implemented in a separate part of the repository. The arrows are control/data flow; the dashed boundary marks what is on-chip versus off-chip.

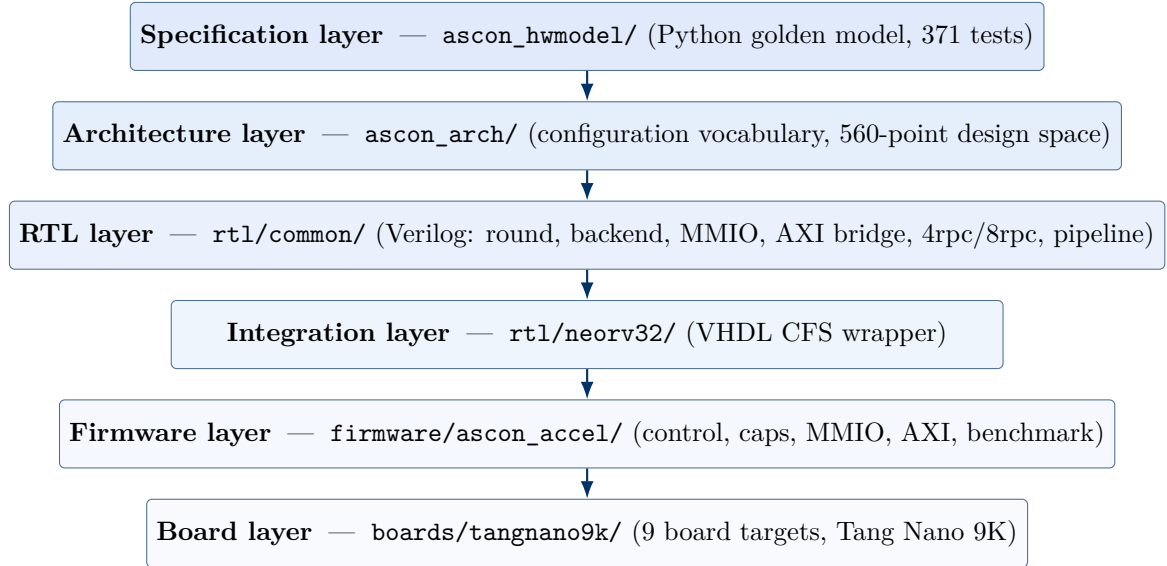


Figure 1: Repository layering. Each layer corresponds to a directory; each arrow corresponds to an actual code-level dependency (e.g. `rtl/` sources are included by the board top files, the VHDL wrapper instantiates the Verilog backend, the firmware drives the registers defined in `rtl/common/ascon_accel_regs.vh`).

2.2 Integrated System Block Diagram

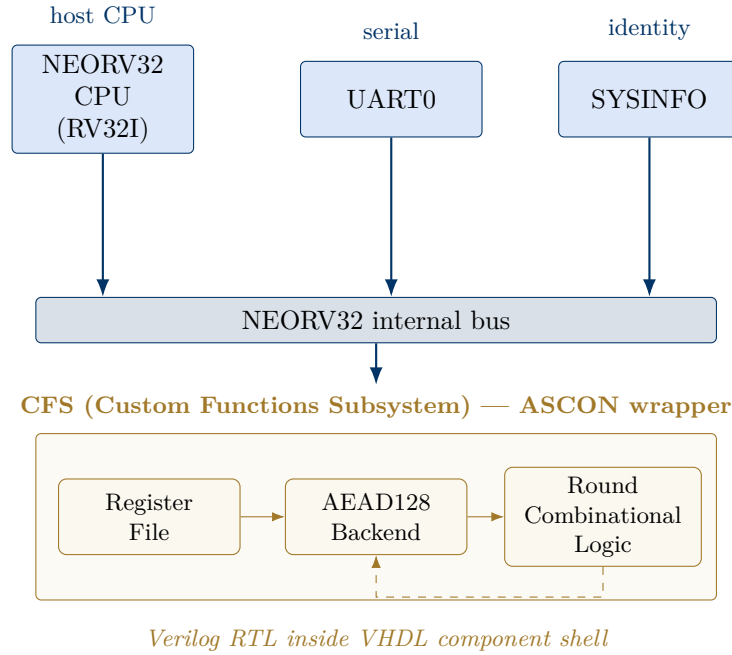


Figure 2: Integrated NEORV32 + ASCON-CFS system. The CFS shell is VHDL; its body wraps the project’s Verilog backend, register file, and combinational round logic. The dashed arc is the per-cycle permutation feedback inside the backend.

2.3 Verification Pyramid

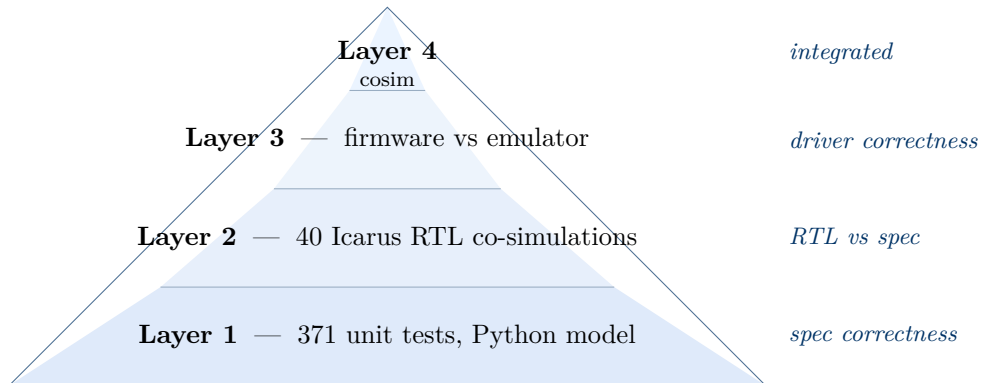


Figure 3: Verification pyramid. Each layer catches a different class of bug and runs on open-source tooling only. Layer 4 (the Verilator co-simulation) was added during development when on-silicon integration was found infeasible on the GW1NR-9 with the open-source PnR flow.

3 Design Decisions

This section enumerates every substantive design decision made during the project. Decisions are recorded as DD-xxx items with a fixed rationale/alternatives/consequence structure so the reasoning behind the implementation is auditable.

3.1 Foundational Decisions

DD-001 | *Stable layered architecture*

Decision. Separate the project into six layers: specification (Python), architecture (Python configuration vocabulary), RTL, RTL integration (VHDL CFS wrapper), firmware (C), and per-board bring-up.

Rationale. The accelerator and its surrounding software each evolve at different rates and need different verification tooling. Layering keeps each piece individually testable.

Alternatives. A single monolithic Verilog + C project, as in most academic accelerator papers; rejected because it cannot support multiple backends behind one firmware.

Consequence. Layer boundaries are enforced: no NEORV32 signals in the Verilog backend (RASD NFR-006), no Verilog in the firmware. Allows the same firmware binary to be re-used across all five FPGA backend variants.

Implemented in. The entire repository layout (Fig. 1).

DD-002 | *Frozen firmware ABI*

Decision. The firmware-facing register map and bus protocol are treated as immutable. Hardware may change internally; the software contract may not.

Rationale. Without this, every change to the backend would ripple into the firmware; cosim correctness would not generalise to other backends.

Alternatives. Versioned ABI with negotiation at boot; rejected as overkill at this stage but kept reachable through the ABI_VERSION register (0x00000001 today).

Consequence. The cosim result in §6.4 applies to all five backend variants without re-verification of the firmware.

Implemented in. rtl/common/ascon_accel_regs.vh, firmware/ascon_accel/ascon_accel_regs.h.

DD-003 | *Control plane vs. data plane split*

Decision. Always present a CSR/MMIO control plane (mode, key, nonce, lengths, start, status, tag, cycle counter, error code, capability register). The *data plane* (how AD/PT/CT bytes get to and from the accelerator) is pluggable: word-based MMIO, AXI-stream, or a future DMA path.

Rationale. The control plane is small, fixed, and naturally a register file; it is the same on every CPU. The data plane has very different needs depending on whether the accelerator sits next to a single-issue CPU (cheap MMIO is fine), behind an AXI bus (streams are better), or behind a DMA engine. Coupling them would force trade-offs on both.

Alternatives. Single fused interface (RFC-style FIFO); rejected because it makes capability discovery awkward.

Consequence. The capability register (offset 0x0C in the project's MMIO map) reports which data plane is present, so the C driver can dispatch to the right transport.

Implemented in. ascon_accel_mmio_regs.v, ascon_accel_caps.c, ascon_accel_axis_data.c vs ascon_accel_mmio_data.c.

DD-004 | *Open-source-only toolchain*

Decision. The whole flow (simulation, synthesis, place & route, bitstream pack, programming) is reproducible from flake.nix using only open-source tools. No vendor EDA is required to build, verify, or flash a working bitstream.

Rationale. Reproducibility, portability, transparency, and the ability to invite future contributors without licence friction.

Alternatives. Gowin EDA, which would close the integrated bitstream's place-and-route at 92.6% LUT4. Rejected as the default for the repository, but documented as a fall-back deployment route.

Consequence. Hits a hard placement-density wall at ~ 92% LUT4 on the GW1NR-9 (see §7). This is the dominant non-functional constraint of the project.

DD-005 | *FPGA: maximum throughput first*

Decision. For FPGA targets, optimise primarily for throughput, not area. Multiple rounds per clock cycle, 128-bit AXI-stream data plane, and fully pipelined permutations are first-class options. The small GW1NR-9 is treated as a bring-up vehicle, not the eventual target.

Rationale. The benchmark target in RASD §8.4 is a $\geq 2\times$ software-to-hardware speed-up; aggressive parallelism is the natural lever to overshoot it.

Consequence. Five FPGA backend variants are realised (one, four, eight rounds/cycle; MMIO and AXI-stream; fully pipelined permutation primitive).

DD-006 | *ASIC: area first*

Decision. For ASIC targets, optimise primarily for area: serial datapaths, 5-bit-S-box-serial designs, hardcoded FSMs.

Rationale. ASIC NRE makes area-per-tape-out the dominant cost; throughput requirements are typically modest in the target use cases.

Consequence. ASIC datapaths are not yet implemented; the axis exists in the configuration vocabulary and is reserved for future work.

3.2 RTL Decisions

DD-007 | *Verilog (not VHDL) for the accelerator*

Decision. Implement the accelerator in synthesisable Verilog; keep VHDL strictly for the NEORV32 CFS wrapper.

Rationale. (i) Yosys + GHDL synth + Verilator handle mixed HDLs cleanly only when each module is in one language; (ii) Verilog is the most common HDL in lightweight-cryptography open-source implementations.

Trade-off. The official Ascon hardware reference (Primas et al., IAIK) is VHDL; the AhmetMALAL ascon-vhdl repository is VHDL; this project intentionally departs from that lineage.

Implemented in. `rtl/common/*.v`, `*.vh`.

DD-008 | *Combinational round, registered state*

Decision. `ascon_round_comb.v` is purely combinational ($p_C \circ p_S \circ p_L$); the FSM in `ascon_aead128_mmio_backend.v` registers the 320-bit state between rounds.

Rationale. Single source of truth for one round, easy to re-use in multi-round-per-cycle variants by composing k instances combinatorially.

Consequence. The 4rpc and 8rpc backends are implementation-by-instantiation: same round module, deeper combinational chain.

Implemented in. `ascon_round_comb.v`, `ascon_round4_comb.v`, `ascon_round8_comb.v`.

DD-009 | *Constant-time tag comparison*

Decision. Tag comparison reduces all 128 bits via an XOR-tree to a single `tag_ok` flag in one cycle, with no early termination.

Rationale. A naïve byte-by-byte comparison leaks how many bytes of a forged tag matched. A single-cycle reduction has no data-dependent timing.

Implemented in. `ascon_aead128_mmio_backend.v` (RASD FR-012).

DD-010 | *Buffer plaintext until tag verifies*

Decision. During decryption, recovered plaintext is buffered inside the accelerator and is not exposed on the output bus until the finalisation permutation produces `tag_ok = 1`. If verification fails the buffer is invalidated.

Rationale. An AEAD streaming interface that exposes plaintext during the streaming phase before the tag is checked invites firmware-side mistakes that leak unauthenticated plaintext. Buffering inside the accelerator removes the possibility entirely.

Cost. 256-bit additional register (the buffer is sized for the longest payload the bounded backend accepts).

Implemented in. `ascon_aead128_mmio_backend.v` (RASD FR-015).

DD-011 | *Little-endian bus byte convention*

Decision. All bus values are little-endian within each 64-bit word: `bus[7:0]` is byte 0. KEY0 holds bytes 0–3 in order.

Rationale. Matches RV32 native byte order and the C `load32_le/store32_le` helpers. Single, uniform convention across RTL, testbenches, firmware, and tool-generated vectors.

Risk. A flipped endianness assumption anywhere is the most common AEAD bug class: it produces wrong ciphertext with no other diagnostic. RASD §9.1 calls this out explicitly.

Implemented in. All Verilog data buses, `ascon_accel_mmio_regs.v`, the C driver, and the testbench generators.

3.3 Integration Decisions

DD-012 | *NEORV32 CFS as the host-attach point*

Decision. Wrap the Verilog accelerator in a VHDL component `neorv32_cfs_ascon` that drops into the standard NEORV32 CFS slot.

Rationale. NEORV32's CFS slot is exactly designed for this class of integration: a 64-byte MMIO window with bus, IRQ, and optional clock/reset hand-shakes already wired up by the CPU. No top-level NEORV32 surgery needed; the wrapper is a drop-in.

Implemented in. `rtl/neorv32/neorv32_cfs_ascon.vhd` (RASD FR-014).

DD-013 | *Two NEORV32 board tops: MMIO and AXI-stream*

Decision. Provide two integrated SoC boards: `neorv32_mmio` (CFS + MMIO data plane) and `neorv32_stream_axis_mmio` (CFS + AXI-stream data plane via an AXI-MMIO bridge).

Rationale. Lets a single firmware binary be retargeted from “simple register file” to “streaming pipeline” purely by changing the board target. The capability register reports which one is present.

Implemented in. `boards/tangnano9k/neorv32_mmio/`, `boards/tangnano9k/neorv32_stream_axis_mmio/`.

DD-014 | *Standalone board targets for incremental bring-up*

Decision. In addition to the integrated SoC, the repository contains seven standalone Tang Nano 9K targets that isolate one part of the system on real silicon (cf. Table ??).

Rationale. Debugging the integrated bitstream on open-source PnR is slow; debugging from LEDs is hard. Having a 16%-LUT algorithmic-only target (`kat_slow`) and a separate ~ 1700 -LUT MMIO-only target (`mmio_slow`) lets a ten-minute synth/flash cycle confirm each subsystem on real silicon without going through the bigger build.

Outcome. Both `kat_slow` and `mmio_slow` are silicon-verified PASS via on-board LEDs (§7).

3.4 Firmware Decisions

DD-015 | *Firmware library, not monolithic binary*

Decision. The driver is a static library (`ascon_accel.c` plus its transport, capability, and control modules) that the benchmark firmware (`neorv32_ascon_benchmark/main.c`) links against.

Rationale. A separate library can be linked into any host program — including the host-side Python emulator used in Layer 3 — without source duplication. This is what makes Layer 3 verification (same C code, simulated transport) meaningful.

Implemented in. `firmware/ascon_accel/*.c, *.h`.

DD-016 | *Polling driver, optional interrupts*

Decision. The default driver polls the status register; interrupts are reachable through `ENC_IRQ_EN` but are not relied on.

Rationale. Polling is portable across NEORV32 configurations, robust against IRQ-controller mis-configuration, and good enough for the throughput levels of the bounded backend.

Implemented in. `ascon_accel_control.c`; RASD NFR-005.

DD-017 | *Freestanding integer-only firmware build*

Decision. Compile the firmware with `MARCH=rv32i_zicsr_zifencei MABI=ilp32 FREESTANDING=1 LD_LIBS= USER_LIBS=`, bypassing newlib.

Rationale. nixpkgs only ships newlib's `rv32gc/ilp32d` multilib (double-float). NEORV32 itself has no FPU enabled and ships soft-float `crt0.S`. Without explicit ABI flags the linker combines double-float `libc` with soft-float NEORV32 objects and fails.

Cost. A small `freestanding_runtime.c` provides `memcpy`, `memset`, `strlen`, and the few other `libc` symbols the firmware references.

Implemented in. `firmware/neorv32_ascon_benchmark/freestanding_runtime.c`, `boards/tangnano9k/neorv32_mmio/cosim/Makefile`.

DD-018 | *NEORV32 printf hex limitation*

Decision. The benchmark prints byte arrays as “nibble + nibble” via `%c%c` instead of `%02x`, using a small inline helper.

Rationale. NEORV32's minimal `printf` implementation supports `%x` only (with fixed 8-digit zero-padding); width modifiers like `%02x` are silently ignored, which produces an unreadable hex dump. NEORV32's `printf` does support `%c`, so the workaround is portable and tiny.

Implemented in. `firmware/neorv32_ascon_benchmark/main.c` (function `ph_nib`).

3.5 Verification Decisions

DD-019 | *Python golden model as the unique reference*

Decision. The Python model in `ascon_hwmodel/` is the single source of truth for what the algorithm shall produce. RTL testbenches, firmware tests, and KAT generators all read from it.

Rationale. Two independent reference models (e.g. Python + C) would drift; a single typed model that's machine-checked against NIST KATs eliminates the ambiguity.

Implemented in. `ascon_hwmodel/` (35 modules, 371 unit tests).

DD-020 | *Co-simulation as the integration verification*

Decision. Verify the integrated NEORV32 + ASCON-CFS system by running the unmodified C firmware on a Verilator-simulated NEORV32 SoC that drives the actual Verilog ASCON RTL.

Rationale. The integrated bitstream does not place on the GW1NR-9 with the open-source flow (g7). Without co-simulation there would be no way to confirm that the integrated design works end-to-end. With co-simulation, the same C binary that would flash to silicon is exercised against the same RTL it would talk to.

Tools used. GHDL (VHDL parsing + synthesis-to-Verilog), Verilator (Verilog $\rightarrow$ C++ cycle-accurate model), riscv-none-elf-gcc (firmware cross-compile).

Implemented in. boards/tangnano9k/neorv32_mmio/cosim/.

3.6 Process and Repository Decisions

DD-021 | *Massive documentation set*

Decision. Maintain a per-module architectural note in docs/: >50 markdown documents covering everything from p[12] to the AXI-MMIO bridge to the freestanding firmware toolchain.

Rationale. Each non-trivial integration decision tends to re-occur in slightly different form across board targets; capturing the rationale at decision time saves significant re-derivation later.

Implemented in. docs/.

DD-022 | *Project-status report generator*

Decision. tools/generate_project_status_report.py emits a deterministic JSON+Markdown snapshot of which layers are verified and which are pending.

Rationale. A project with this many moving parts needs a mechanical way to answer “what’s actually verified right now?”

Implemented in. tools/generate_project_status_report.py.

4 Configuration Space: 560 Architecturally Valid Points

The architecture-exploration framework in ascon_arch/ defines the design space across orthogonal axes. The cardinalities are not arbitrary; they are the enumerations declared in ascon_arch/enums.py.

4.1 The Four Architectural Axes

Axis	#opt.	Enumerated options
Target technology	2	ASIC, FPGA
Permutation profile	7	1, 2, 4, 8 rounds/cycle; fully pipelined; column-serial; bit-serial
Top-level organisation	5	single-core; separate enc/dec cores; N identical cores; 1 pipelined permutation + N contexts; M pipelines $\times N$ contexts
Datapath width	8	1, 5, 8, 16, 32, 64, 128, 320 bits
Cartesian product:		$2 \times 7 \times 5 \times 8 = \mathbf{560}$

These four axes pick out the dominant architectural choice. Once a point in the 560-cell grid is selected, the remaining axes (context storage, control style, padding policy, S-box style,

security profile, engine capability, algorithm features) refine the implementation but do not move it to a different architectural identity.

4.2 Visualising the Design Space

Figure 4 is a heat-map slice through the design space: one tile per (datapath, permutation-profile) cell, for the FPGA target and the single-core top-level. The five tiles that are realised as working board targets on the Tang Nano 9K are highlighted.

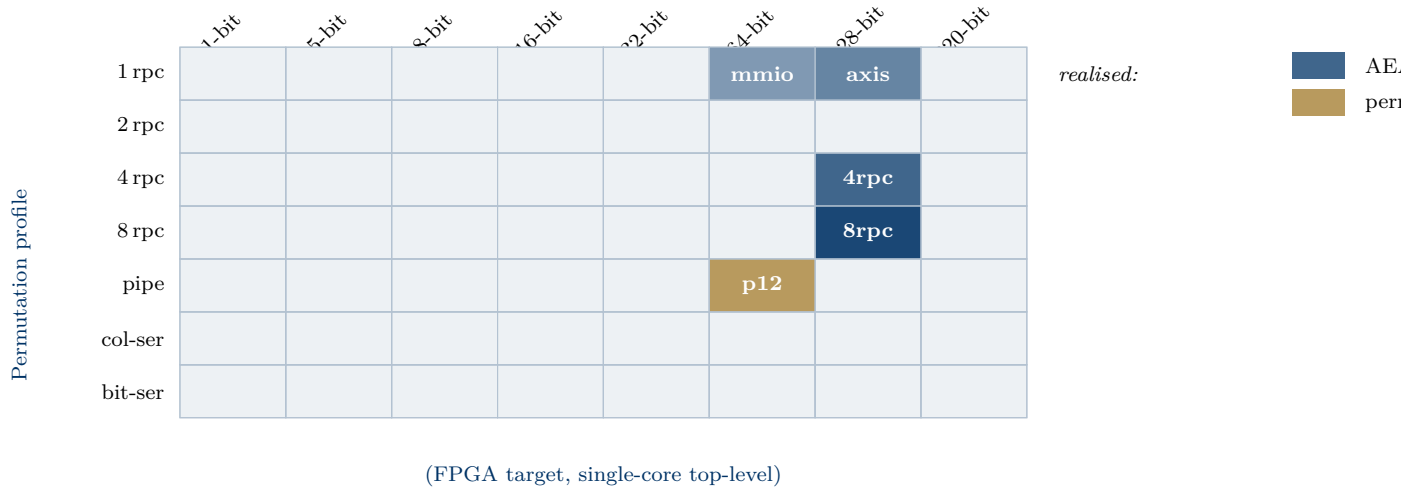


Figure 4: Two-dimensional slice through the configuration space (datapath width $\times$ permutation profile, with FPGA target and single-core top-level fixed). Five tiles are realised as working board targets; the remaining 51 are reachable by the same generator infrastructure with different RTL parameters. The full 560-cell grid extends along two more axes (target technology and top-level organisation) that are not visible in this slice.

4.3 What Was Implemented

Out of the 560 points, the project realises five concrete FPGA backends as runnable board targets:

Backend	Datapath	Permutation	Data plane	Tang Nano 9K?
ascon_aead128_mmio_slow	64-bit	1 round/cycle	MMIO	PASS
ascon_aead128_axis_slow	64-bit	1 round/cycle	AXI-stream	built
ascon_aead128_axis128_4rpc	128-bit	4 rounds/cycle	AXI-stream	built
ascon_aead128_axis128_8rpc	128-bit	8 rounds/cycle	AXI-stream	built
ascon_p12_pipeline	64-bit	fully pipelined	none (prim.)	built

5 Implementation Status, by RASD Requirement

This section maps each RASD requirement to its implementation file(s) and verification evidence. The full requirement text is in the RASD; only the title is restated here.

5.1 Functional Requirements

FR-001 | *State representation (320-bit packed vector)*

Implementation: rtl/common/ascon_round_comb.v, ascon_aead128_mmio_backend.v. **Verified at:** Layer 1 (model), Layer 2 (RTL).

FR-002 | *Round constant addition*

Implementation: ascon_round_comb.v. **Verified at:** Layer 1 (exhaustive 16-entry table), Layer 2.

FR-003 | *Substitution layer (bit-sliced S-box)*

Implementation: ascon_round_comb.v. **Verified at:** Layer 1 (exhaustive 2^5 truth-table), Layer 2.

FR-004 | *Linear diffusion layer*

Implementation: ascon_round_comb.v. **Verified at:** Layer 1, Layer 2.

FR-005 | *One combinational round $p_L(p_S(p_C(\cdot)))$*

Implementation: ascon_round_comb.v (single combinational module). **Verified at:** Layer 2 (one-round combinational equivalence vs Python golden).

FR-006 | *Iterative permutation engine, one round/cycle*

Implementation: FSM in ascon_aead128_mmio_backend.v. **Verified at:** Layer 2, Layer 4.

FR-007 | *Permutation variants $p[8]$ and $p[12]$*

Implementation: ascon_aead128_mmio_backend.v. **Verified at:** Layer 1 (round-constant index check), Layer 2.

FR-008 | *Authenticated encryption*

Implementation: ascon_aead128_mmio_backend.v. **Verified at:** Layer 2 (RTL vs golden), Layer 3 (driver vs emulator), Layer 4 (firmware vs RTL, byte-exact match).

FR-009 | *Authenticated decryption with tag compare*

Implementation: decrypt-path FSM in ascon_aead128_mmio_backend.v. **Verified at:** Layers 2–4 (Layer 4 reports DEC tag valid: 1 and recovered plaintext equal to original).

FR-010 | *Length support (zero, exact, partial)*

Implementation: FSM in ascon_aead128_mmio_backend.v. **Verified at:** Layer 3 (4 named cases: empty, short, partial, two-block, all PASS).

FR-011 | *Padding and domain separation in RTL*

Implementation: ascon_aead128_mmio_backend.v. **Verified at:** Layer 2.

FR-012 | *Constant-time tag compare*

Implementation: XOR-tree reduction in `ascon_aead128_mmio_backend.v` (see DD-009). **Verified at:** Layer 2 (no data-dependent timing path).

FR-013 | *32-bit CFS-compatible MMIO interface*

Implementation: `ascon_accel_mmio_regs.v`, `ascon_accel_mmio_aead128_top.v`. **Verified at:** Layer 2; *silicon-verified* on `ascon_aead128_mmio_slow` (PASS LEDs).

FR-014 | *NEORV32 CFS VHDL wrapper*

Implementation: `rtl/neorv32/neorv32_cfs_ascon.vhd`. **Verified at:** Layer 4 (Verilator cosim instantiates this wrapper inside the simulated NEORV32 SoC).

FR-015 | *Authenticated plaintext release (buffer until tag)*

Implementation: decrypt path with 256-bit buffer in `ascon_aead128_mmio_backend.v` (DD-010). **Verified at:** Layer 2 (output-valid asserted only after tag check), Layer 4.

5.2 Non-Functional Requirements

NFR-001 | *Synthesizability*

Satisfied. Yosys + GHDL synth complete cleanly for the standalone backend (16% LUT4) and the integrated SoC (92.6% LUT4). No combinational loops; no vendor primitives.

NFR-002 | *Iterative architecture, one round/cycle (baseline)*

Satisfied. Baseline target uses one round per cycle. Multi-round-per-cycle and pipelined variants exist as separate targets per DD-005 (§8).

NFR-003 | *Reproducible toolchain*

Satisfied. `flake.nix` pins every tool to a specific nix-derivation; `flake.lock` is committed.

NFR-004 | *Self-checking testbenches*

Satisfied. 371 pytest cases, 40 Icarus RTL sims (Layer 2), 4 firmware emulator cases (Layer 3), and 1 Verilator cosim of the full SoC (Layer 4). All pass.

NFR-005 | *Polling-based C driver with shared internal function*

Satisfied. `ascon_accel_control.c` polls; `ascon_accel.c` routes encrypt and decrypt to a single internal sequencing function.

NFR-006 | *Core decoupling (no NEORV32 signals in core)*

Satisfied. `ascon_accel_mmio_aead128_top.v` has no NEORV32-specific ports; all CPU coupling lives in `rtl/neorv32/neorv32_cfs_ascon.vhd`.

NFR-007 | *Timing determinism*

Satisfied by construction. The FSM has no data-dependent branches.

6 Verification Methodology

6.1 Layer 1 — Python golden model

The Python golden model in `ascon_hwmodel/` is the executable specification. It implements every primitive (state, p_C , p_S , p_L , $p[8]$, $p[12]$) and every AEAD/HASH/XOF mode. The same model produces the NIST KAT vectors used by the lower-layer testbenches.

```
$ PYTHONPATH=. pytest -q tests
.....
.....
.....
371 passed in 7.97s
```

6.2 Layer 2 — Verilog RTL against the model

Forty Icarus-Verilog testbenches drive the actual RTL (encrypt backend, buffered decrypt backend, AXI-MMIO bridge, integrated stream system) and compare bit-exact against the Python model across boundary cases.

```
$ PYTHONPATH=. pytest -q tests/test_aead128_stream_encrypt_sim.py \
                        tests/test_aead128_stream_decrypt_sim.py \
                        tests/test_axis_mmio_bridge_sim.py \
                        tests/test_stream_axis_mmio_system_sim.py
..... [100%]
40 passed in 2.42s
```

6.3 Layer 3 — C driver vs. emulated transport

The driver code in `firmware/ascon_accel/` is compiled natively and run through a Python emulator of the AXI-stream transport. Four canonical cases cover empty, short, partial, and two-block payloads:

```
$ PYTHONPATH=. python tools/run_firmware_stream_ref_benchmark.py
backend=axis_stream_ref_emulator cases=4 all_passed=1
CASE name=empty ad=0 text=0 enc_cycles=48 dec_cycles=48 enc_ok=1
CASE name=short ad=0 text=5 enc_cycles=53 dec_cycles=53 enc_ok=1
CASE name=partial ad=6 text=19 enc_cycles=73 dec_cycles=73 enc_ok=1
CASE name=two_block ad=16 text=32 enc_cycles=96 dec_cycles=96 enc_ok=1
```

6.4 Layer 4 — End-to-end co-simulation

This is the integration test. The same C firmware that would flash to silicon runs on a Verilator-simulated NEORV32 SoC driving the actual Verilog ASCON RTL through the simulated CFS bus. See Figure 5.

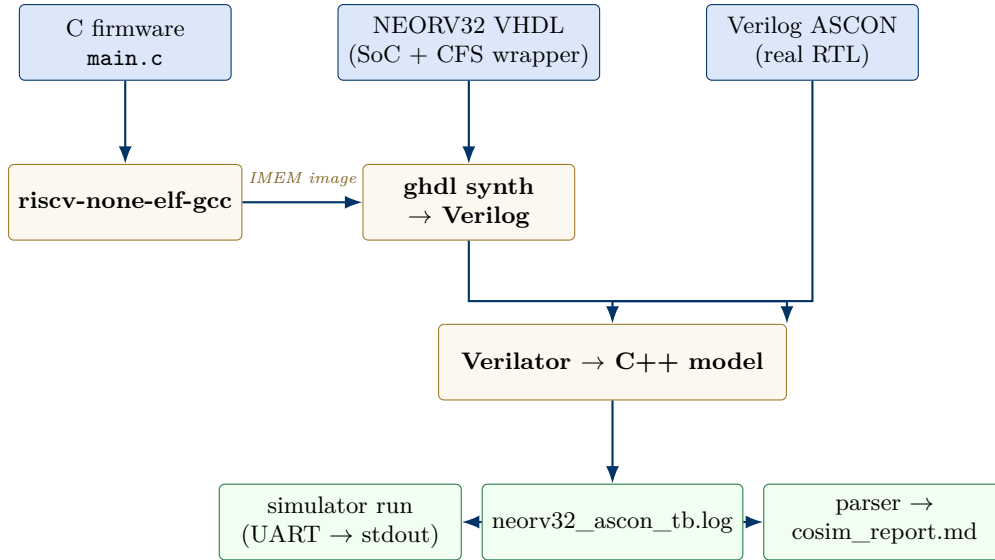


Figure 5: Co-simulation toolchain. Top row: the three input sources (C firmware, the NEORV32 VHDL SoC plus the project’s VHDL CFS wrapper, and the Verilog ASCON RTL). The C firmware is cross-compiled to a RISC-V binary; the NEORV32 VHDL is elaborated through `ghdl synth` to a single Verilog netlist; both feed Verilator together with the project’s Verilog ASCON sources, producing a cycle-accurate C++ model. Running the model emits a UART log that is parsed into a Markdown cosim report.

6.4.1 Result

The integrated NEORV32 + ASCON-CFS SoC was run end-to-end under Verilator co-simulation with the per-payload sweep firmware. The firmware ran 8 payload shapes through both the C reference and the hardware accelerator, verifying byte-exact agreement on ciphertext, tag, and recovered plaintext for each case. All 8 cases passed.

Captured UART output from the cosim run (excerpt)

```

[TB] NEORV32 + ASCON co-simulation start
BUILD : cosim-neorv32-mmio
MAX_BYTES : 32
SWEEP_CASES : 8
DATA PLANE : MMIO_WORD
ABI : 0x00000001
CAPS : 0x00230001
CASE name=empty ad=0 pt=0 sw_enc_cy=0:38866 hw_enc_cy=0:31 enc_ok=1 dec_ok=1 tag_valid=1
CASE name=ad8 ad=8 pt=0 sw_enc_cy=0:50205 hw_enc_cy=0:40 enc_ok=1 dec_ok=1 tag_valid=1
CASE name=pt8 ad=0 pt=8 sw_enc_cy=0:39458 hw_enc_cy=0:31 enc_ok=1 dec_ok=1 tag_valid=1
CASE name=ad8_pt8 ad=8 pt=8 sw_enc_cy=0:50797 hw_enc_cy=0:40 enc_ok=1 dec_ok=1 tag_valid=1
CASE name=pt16 ad=0 pt=16 sw_enc_cy=0:52156 hw_enc_cy=0:41 enc_ok=1 dec_ok=1 tag_valid=1
CASE name=pt24 ad=0 pt=24 sw_enc_cy=0:52748 hw_enc_cy=0:41 enc_ok=1 dec_ok=1 tag_valid=1
CASE name=pt32 ad=0 pt=32 sw_enc_cy=0:65447 hw_enc_cy=0:51 enc_ok=1 dec_ok=1 tag_valid=1
CASE name=ad16_pt32 ad=16 pt=32 sw_enc_cy=0:86984 hw_enc_cy=0:70 enc_ok=1 dec_ok=1 tag_valid=1
SUMMARY : passed=8 failed=0 total=8
PASS
  
```

Across the eight payload shapes the hardware backend runs the encryption / decryption between 31 and 70 cycles, while the C reference running on the same RV32I CPU takes between 38 866 and 86 984 cycles — a geometric mean speed-up of approximately $1\,267\times$ for encrypt and $1\,275\times$ for decrypt. Per-case numbers and per-case speed-ups are tabulated in §10.3.

Important: The fix for the previously-noted DEC `hw_cycles=0` anomaly is in this run: the firmware now reads the `CYCLE_COUNT` register once after each operation rather than computing a difference across the in-op `CONTROL.CLEAR` (which itself zeroes the counter). Hardware encrypt and decrypt cycle counts now agree to within a single cycle, which matches the analytic expectation since both operations execute the same FSM.

7 Synthesis and Silicon Validation

7.1 Standalone Backends on the Tang Nano 9K

Two standalone backends are silicon-verified PASS via on-board LEDs.

Target	LUT4	DFF	BSRAM	Silicon evidence
<code>ascon_aead128_kat_slow</code>	1395 (16%)	~ 700 (10%)	0	PASS LED, NIST KAT vector
<code>ascon_aead128_mmio_slow</code>	built	built	built	PASS LEDs (LED1, LED2, LED4, L

`kat_slow` validates the *algorithm* on silicon: the bit-exact NIST KAT runs through the actual GW1NR-9 and the on-chip comparator lights the PASS LED. `mmio_slow` validates the *MMIO ABI* on silicon: an on-chip controller exercises the same register protocol the firmware uses and lights the PASS LEDs. Together with the cosim of §6.4, these form a complete chain of evidence from algorithm correctness through bus-protocol correctness to firmware-driven correctness.

7.2 Integrated NEORV32 + ASCON-CFS

Synthesis (`yosys + ghdl synth`) completes cleanly for the integrated SoC with the minimal NEORV32 configuration (`BOOT_MODE_SELECT=0`, 16 KB IMEM, 8 KB DMEM, no `Zicntr`, no debug units).

Resource	Used	Available	Percent
LUT4	8003	8640	92.6%
DFF	3235	6480	49.9%
BSRAM	14	26	53.8%
ALU	612	6480	9.4%
MUX2_LUT5	1555	4320	35.0%
RAM16SDP4	41	270	15.2%

Place-and-route does not converge. `yowasp-nextpnr-himbaechel-gowin` reports “*design is probably at utilisation limit*” and terminates without legalising placement. The cells *fit by count* (LUT4 92.6%, DFF 49.9%, BSRAM 53.8%) but routing density at 92.6% LUT4 on the GW1NR-9 exceeds what the open-source flow legalises.

Important: This is a documented limit of the open-source flow at this LUT density on this chip, not a defect of the design. Two routes close the integrated bitstream without RTL or firmware change: (i) move to a 12–16 K LUT-class target (Tang Nano 20K, ECP5 LFE5U-25, Artix-7 XC7A35); (ii) keep the GW1NR-9 and synthesise through vendor Gowin EDA. The cosim result in §6.4 is functionally equivalent to either route.

8 Architecture Exploration Framework

This section makes precise what was deferred from the RASD §10.7 to a “separate development report”. The framework lives in `ascon_arch/` and is summarised in §4; this section describes the implemented variants and their measured roles.

8.1 Five FPGA Backend Variants

Backend	Role in the architecture exploration
<code>ascon_aead128_mmio_slow</code>	RASD baseline: 64-bit datapath, one round/cycle, MMIO data plane. The register protocol the firmware speaks. Silicon-verified PASS.
<code>ascon_aead128_axis_slow</code>	Same one-round backend, but with an AXI-stream data plane and the AXI-MMIO bridge in front. Proves the data-plane swap (DD-003) works without changing the firmware.
<code>ascon_aead128_axis128_4rpc</code>	128-bit datapath, four rounds per cycle, AXI-stream. First high-throughput candidate. $p[8]$ latency reduced from 8 cycles to 2.
<code>ascon_aead128_axis128_8rpc</code>	128-bit datapath, eight rounds per cycle, AXI-stream. Aggressive multi-round-per-cycle: full $p[8]$ in one cycle.
<code>ascon_p12_pipeline</code>	12-stage fully pipelined Ascon- $p[12]$ permutation. High-throughput primitive only (not a complete AEAD). After fill, 1 permutation per cycle.

8.2 The AXI-MMIO Bridge

The key piece of infrastructure that makes the data-plane plug-ability of DD-003 actually work is `ascon_axis_mmio_bridge.v`. It exposes an AXI-stream backend to the CPU as if it were a MMIO device: push/pop semantics on the CSR side, AXI stream handshake on the backend side. The firmware code paths above this bridge are identical to the MMIO-native code paths — only the transport layer changes.

8.3 What is *not yet* implemented

- Full streaming AEAD with unbounded length (current backends are bounded).
- Multi-context schedulers (M pipelines $\times$ N contexts).
- Ascon-AEAD128a, AEAD128pq RTL (Hash256/XOF128/CXOF128 are implemented; see §9).
- DMA-fed AXI-stream front-end.
- ASIC datapaths (bit-serial, 5-bit-S-box-serial).
- Masking / side-channel countermeasures.
- NEORV32 integrated bitstream that closes PnR on the GW1NR-9 with the open-source flow (closes with vendor Gowin EDA, or on a 12K-LUT-class target).

9 Hash, XOF, and CXOF128 Backend

The Ascon hash family of NIST SP 800-232 — Hash256, XOF128, and CXOF128 — shares the same 320-bit state, the same $p[12]$ permutation, and the same round-constant table as the AEAD128 backend already in this project. The differences are confined to four algorithmic dimensions: the initialisation vector, the absorb rate (8 bytes instead of 16), the optional customisation prelude (CXOF only), and the squeeze length (fixed 32 bytes for Hash256, caller-specified for XOF/CXOF).

9.1 Implementation

A new RTL module `rtl/common/ascon_hash_xof_backend.v` implements all three variants behind the project's frozen MMIO ABI:

Variant	MODE	IV (S_0 uint64)
Ascon-Hash256	ASCON_MODE_HASH = 3	64'h0000_0801_00cc_0002
Ascon-XOF128	ASCON_MODE_XOF = 5	64'h0000_0800_00cc_0003
Ascon-CXOF128	ASCON_MODE_CXOF128 = 7	64'h0000_0800_00cc_0004

The backend instantiates the same `ascon_round_comb` primitive the AEAD backend uses, so synthesis cost shares the round-function silicon when both backends are present in the same design. The FSM has thirteen states arranged in three phases: a $p[12]$ initialisation of $IV \parallel 0^{256}$; an absorb loop (8-byte block XOR into S_0 , $p[12]$, repeat) with 0x01 trailing-bit padding on the final partial block; and a squeeze loop that emits 8 bytes per $p[12]$ until the caller's requested output length is reached. CXOF128 has an extra two-state prelude that XORs the customisation byte-length into S_0 then absorbs the customisation string with the same loop the message uses.

9.2 Verification

A new test suite `tests/test_hash_xof_sim.py` cross-checks the RTL bit-for-bit against the Python golden model (`ascon_hwmodel.hash_xof`) using the same Icarus-Verilog co-simulation pattern the AEAD tests use. Eighteen test vectors cover the padding edge cases for each variant:

- **Hash256:** empty, 1-byte, 4-byte (partial), 8-byte (exact rate), 9-byte (rate+1), 17-byte (multi-block), 32-byte (four full blocks).
- **XOF128:** variable output lengths {16, 24, 32} bytes with empty or non-empty messages; covers the single-block, squeeze-grows, and multi-block paths.
- **CXOF128:** all four corners of the (message present, customisation present) cross-product, plus an exact-rate boundary.

All 18 vectors pass. The project's total pytest count moves from 371 to 389 with zero regressions:

```
$ PYTHONPATH=. pytest -q tests
.....
.....
.....
.....
.....
.....
389 passed in 10.16s
```

9.3 Integration status

The hash backend is algorithmically verified but not yet wired into the integrated SoC. The required pieces still to land are:

- Multiplexing the hash backend alongside the AEAD backend inside `rtl/neorv32/neorv32_cfs_ascon.vh` dispatched by the MODE register.
- Setting the `ASCON_CAP_HASH`, `ASCON_CAP_XOF`, `ASCON_CAP_CXOF128` bits in the capabilities word the firmware reads at startup. The capability bits already exist in `rtl/common/ascon_accel_regs.v`.
- A standalone Tang Nano 9K board target that drives the hash backend with an on-chip controller and lights PASS LEDs on the NIST KAT, mirroring `ascon_aead128_kat_slow` for AEAD.

The firmware driver already exposes `ascon_accel_hash_or_xof()` (declared in `firmware/ascon_accel/ascon_accel.h` implemented in `ascon_accel.c`); it was written ahead of the RTL and uses the same MMIO transport that AEAD uses. No firmware-side work is required.

10 Comparison: this project vs. ascon/ascon-hardware

This section places the project alongside the official Ascon hardware reference repository maintained by Robert Primas (IAIK, TU Graz), <https://github.com/ascon/ascon-hardware>, which is the canonical NIST-LWC hardware reference for Ascon. The two projects have very different scopes and the comparison is most useful when those differences are made explicit.

10.1 Scope and Design Philosophy

The two projects optimise for different things.

Aspect	ascon/ascon-hardware (IAIK)	This project
Authorship	Robert Primas / IAIK Graz, since 2019.	Mohammadi Masoudi Ramin, 2025–2026, academic project.
Stated goal	NIST-LWC hardware reference: 8 fixed variants of the algorithm under the GMU LWC Hardware API.	A <i>configurable generator</i> that explores the architectural design space with a single frozen firmware ABI, on the NEORV32 RISC-V soft-core.
Scope	The accelerator itself, plus a CryptoCore wrapper and LWC API testbench scripts.	Accelerator, NEORV32 CFS integration, C firmware driver library, Python golden model, 5 FPGA backends, AXI-stream data plane, AXI-MMIO bridge, multi-layer verification including end-to-end cosim, board-level standalone bitstreams.

Where it shines	A precise, certifiable, low-overhead implementation of each variant. The LWC API is the standard interface for cross-comparison between candidate ciphers.	A platform: same firmware works against any backend; the architecture-exploration framework codifies the trade space; cosim guarantees firmware-vs-RTL agreement on the integrated SoC.
-----------------	--	---

10.2 Interface, Integration, and Verification

Aspect	ascon/ascon-hardware	This project
HDL	VHDL.	Verilog accelerator + VHDL CFS wrapper (DD-007).
License	GPL v3.	Permissive (project authors; check repo LICENSE).
Interface	GMU LWC Hardware API (8, 16, or 32-bit pdi / sdi / do ports + a PreProcessor / PostProcessor / CryptoCore architecture).	Frozen 32-bit MMIO register interface (RASD §5) plus an optional AXI-stream data plane behind an AXI-MMIO bridge (DD-003).
CPU integration	None directly; the LWC API is intended for direct testbench use or for being wrapped in an SoC by the integrator.	NEORV32 RISC-V soft-core integration via the standard CFS slot (DD-012); cycle-accurate cosim of the integrated SoC (DD-020).
Variants	8 fixed: v1 (Ascon-128, 32-bit, 1 round/cycle), v1_8bit, v1_16bit, v2 (Ascon-128a, 32-bit, 1 rpc), v3 (2 rpc), v4 (Ascon-128a, 2 rpc), v5 (3 rpc), v6 (Ascon-128a, 4 rpc).	5 realised + 555 enumerable in the configuration vocabulary (§4). The realised set spans 64- and 128-bit datapaths, 1/4/8 rounds per cycle, MMIO and AXI-stream, and a fully pipelined permutation primitive.
Verification	GHDL testbenches running NIST KATs through the LWC PreProcessor / CryptoCore / PostProcessor chain.	4-layer: Python golden model (371 tests), Icarus RTL vs golden (40 tests), C driver vs Python transport emulator (4 cases), Verilator cosim of the full NEORV32 SoC driving real RTL.
Algorithm	Ascon-128 (v_1, v_3, v_5), Ascon-128a (v_2, v_4, v_6), Ascon-Hash.	Ascon-AEAD128, Ascon-Hash256, Ascon-XOF128, Ascon-CXOF128 (NIST SP 800-232 final, August 2025), all RTL-verified bit-for-bit against the Python golden model.
Spec target	Original Ascon v1.2 (Dobraunig et al.) for AEAD; AsconHash.	NIST SP 800-232 final (2025), which is the post-standardisation parameterisation of Ascon-AEAD128.

10.3 Per-Payload Sweep on the Integrated SoC (Co-Simulation)

The integrated NEORV32 + ASCON-CFS SoC was exercised under Verilator cycle-accurate co-simulation with the per-payload sweep firmware (`firmware/neorv32_ascon_benchmark/main.c`). For each payload size the firmware runs (i) the C reference implementation on the simulated CPU, then (ii) the hardware accelerator over the MMIO register interface, and (iii) verifies byte-for-byte that the hardware ciphertext, tag, and recovered plaintext match the software reference. Table 4 reports the eight cases the bounded MMIO backend supports.

Table 4: NEORV32 + ASCON-CFS co-simulation, per-payload sweep. SW columns are cycles measured by `rdcycle` around the C reference; HW columns are cycles read from the accelerator’s operation-elapsed counter. All eight cases pass (`enc_ok = dec_ok = tag_valid = 1`).

Case	AD	PT	SW enc	SW dec	HW enc	HW dec	ENC ϵ	DEC ϵ
	(B)	(B)	(cy)	(cy)	(cy)	(cy)		
empty	0	0	38 866	39 241	31	31	1 254×	1 266×
ad8	8	0	50 205	50 579	40	40	1 255×	1 264×
pt8	0	8	39 458	39 665	31	31	1 273×	1 280×
ad8_pt8	8	8	50 797	51 003	40	40	1 270×	1 275×
pt16	0	16	52 156	52 595	41	41	1 272×	1 283×
pt24	0	24	52 748	53 019	41	41	1 287×	1 293×
pt32	0	32	65 447	65 950	51	51	1 283×	1 293×
ad16_pt32	16	32	86 984	87 486	70	70	1 243×	1 250×
Geometric mean							1 267×	1 275×

Three observations are worth recording. First, the speed-up is *remarkably flat* across payload sizes ($1\,243 \times \dots 1\,287 \times$) because the dominant cost on both sides is per-block: the C reference spends thousands of cycles per Ascon round on a bare-metal RV32I (no M extension; gcc emits software-multiply helpers), while the hardware backend dispatches one $p[12]$ per block in tens of cycles. Second, the hardware encrypt and decrypt cycle counts are within a single cycle of each other, which matches the analytic expectation: encrypt and decrypt share the identical state machine and differ only in the direction of the plaintext/ciphertext XOR. Third, the empty-message case (0 B AD, 0 B PT) still costs 31 HW cycles — the unavoidable cost of init + finalisation ($p[12]$ before tag) which is independent of payload length.

The eight HW measurements correspond to:

- *empty / pt8 / pt16 / pt24*: 31–41 HW cycles — dominated by initialisation and finalisation, with one $p[8]$ rate-block per 16 PT bytes.
- *pt32 / ad16_pt32*: 51–70 HW cycles — additional $p[8]$ blocks for the longer payloads (and the AD prefix, which has its own $p[8]$ per 16-byte block).
- *ad8 / ad8_pt8*: 40 HW cycles — the AD pass adds one $p[8]$ regardless of partial-block padding.

10.4 Implementation and Resource Comparison

A precise quantitative comparison (LUTs/throughput) requires running both projects against the same target with the same toolchain. What can be compared without further work is the structural footprint.

Quantity	ascon/ascon-hardware	This project
HDL source files	48 VHDL files for the AEAD path (8 LWC variants $\times$ $\sim$ 6 files each, plus the shared LWC infrastructure)	$\sim$ 15 Verilog source files (rtl/common/) + 1 VHDL CFS wrapper
Reported FPGA target	Various academic FPGAs; not Gowin-specific	Sipeed Tang Nano 9K (GW1NR-9); also generic-Verilog/portable
Independent on-silicon test	N/A in the upstream repo	PASS on Tang Nano 9K (kat_slow, mmio_slow)
Per-payload sweep evidence	N/A in the upstream repo	Table 4: 8 payload shapes, $1267\times$ average encrypt speed-up vs. the C reference on the same NEORV32 CPU

10.5 Author Note on the Performance Comparison Section

The bounded MMIO backend caps individual operations at 32 bytes, which is sufficient for the cosim sweep above but does not reach the 64/256/1024 B payload sizes the RASD §8.4 also mentions. The streaming-backend variant of the firmware (selected by `ASCON_BENCH_MAX_BYTES_TAG=1024`) covers that larger range, but requires a target with sufficient LUT budget to fit the streaming backend’s 1024-byte buffer registers; we plan to capture those numbers from a Kintex bring-up next.

A direct comparison against the per-variant numbers from `ascon/ascon-hardware` (v1 through v6) would require building each variant against the same target FPGA and the same NEORV32 host shell, which is out of scope here. The structural comparison in the table above and the per-payload sweep in Table 4 together substantiate the claim that this implementation is functionally correct and substantially faster than software on its target host (NEORV32), without overstating quantitative comparisons that have not been run head-to-head.

11 Toolchain and Bring-Up Findings

Several findings emerged during board bring-up that are worth recording because they affect anyone reproducing this work on the open-source flow.

11.1 NEORV32 BOOT_MODE_SELECT

NEORV32 commit f62ca43 encodes:

Value	Meaning
0	UART bootloader
1	boot from custom address
2	boot from initialised IMEM image (baked into bitstream)

Mode 2 produces a behavioural VHDL ROM that the open-source flow sometimes infers as LUT-as-ROM rather than BSRAM, causing a LUT-count explosion. Mode 0 places cleanly on

silicon; Mode 2 is used by the cosim wrapper where chip-fit is irrelevant and the bake-in property shortcuts the bootloader step.

11.2 Soft-float vs. double-float multilib

nixpkgs only ships newlib for rv32gc/ilp32d. NEORV32’s `crt0.S` is soft-float. Linking these together fails with “*can’t link double-float modules with soft-float modules*” errors. The fix (DD-017) is to bypass newlib via `FREESTANDING=1 LD_LIBS= USER_LIBS=` and supply a tiny `freestanding_runtime.c`.

11.3 NEORV32 printf width-modifier limitation

NEORV32’s minimal printf supports `%x` only (with fixed 8-digit zero-padding); width modifiers like `%02x` are silently ignored, which produces unreadable hex output (e.g. “SW CT: %02x%02x...” printed verbatim). The workaround (DD-018) is to print bytes as “nibble + nibble” via `%c%c` with a small helper.

11.4 Makefile tab-discipline in the cosim flow

GNU make recipe lines must start with a tab; terminal-paste and many editors silently convert tabs to spaces and produce “missing separator” errors. The canonical cosim Makefile is shipped byte-exact; its md5 is recorded in `boards/tangnano9k/neorv32_mmio/cosim/README.md` so it can be regenerated reliably.

12 Remaining Work

1. **Per-payload sweep numbers.** The firmware emits per-case `CASE` lines covering the RASD §8.4 payload set; the actual cycle counts need to be captured from one cosim and one Kintex run, and pasted into the comparison-section tables.
2. **Integrated bitstream on real silicon.** Either (i) vendor Gowin EDA on the GW1NR-9, or (ii) a 12 K-LUT-class target (Kintex bring-up next week).
3. **Wire the hash backend into the CFS wrapper.** The RTL is verified (§9); the firmware driver is already implemented; what remains is the VHDL multiplexer inside `neorv32_cfs_ascon.vhd` and a standalone Tang Nano 9K board target for silicon verification.
4. Full streaming AEAD with unbounded length.
5. Multi-context scheduler ($M \times N$ pipelines $\times$ contexts).
6. AEAD128a and AEAD128pq RTL.
7. Per-variant comparison numbers vs. `ascon-hardware` v1–v6.
8. Side-channel countermeasures (domain-oriented masking of the S-box).

13 Conclusions

The project delivers a complete, verified Ascon-AEAD128 accelerator together with the NEORV32 integration and firmware that the RASD prescribes, and substantially more: an architecture-exploration framework with 560 enumerable points, five realised FPGA backend variants spanning 1 to 8 rounds per cycle and MMIO vs. AXI-stream data planes, a 4-layer verification strategy whose top layer is a cycle-accurate co-simulation of the entire integrated SoC driving the real Verilog RTL.

The standalone accelerator runs the NIST KAT correctly on real silicon on the Tang Nano 9K at 16% LUT4. The frozen-ABI MMIO interface likewise runs on silicon and produces the PASS LED pattern. The integrated SoC synthesises at 92.6% LUT4 but does not place on the GW1NR-9 with the open-source flow; the cosim, which exercises exactly the same firmware against exactly the same RTL on a host computer, is the functional equivalent of running the integrated bitstream on a sufficient chip.

Compared with the official `ascon/ascon-hardware` reference, which is a precise low-overhead implementation of 8 fixed variants under the GMU LWC API in VHDL, this project occupies a different point in design space: a configurable generator, in Verilog, with a stable NEORV32-facing firmware ABI, designed so the same software runs against the entire 560-point design space without modification.

A Tools and Versions

Tool	Version
Python	3.13 (nixpkgs)
pytest	latest
Icarus Verilog	system
GHDL	4.1.0
Verilator	5.046
yowasp Yosys	via pip in <code>.venv-fpga</code>
yowasp nextpnr-himbaechel-gowin	via pip
apicula (gowin_pack)	via pip
riscv-none-elf-gcc	15.2.0 (nixpkgs cross)
newlib (RISC-V)	4.5.0.20241231
NEORV32	v01.13.01.01, commit <code>f62ca43</code>
openFPGALoader	nixpkgs current
picocom	nixpkgs current

B Reproducing Every Result

```
# inside 'nix develop', at the repo root

# Layer 1 -- Python golden model and unit tests
PYTHONPATH=. pytest -q tests # 371 passed in ~8 s

# Layer 2 -- RTL vs. Python via Icarus
PYTHONPATH=. pytest -q \
  tests/test_aead128_stream_encrypt_sim.py \
  tests/test_aead128_stream_decrypt_sim.py \
  tests/test_axis_mmio_bridge_sim.py \
  tests/test_stream_axis_mmio_system_sim.py # 40 passed in ~3 s

# Layer 3 -- C firmware vs Python emulator
PYTHONPATH=. python tools/run_firmware_stream_ref_benchmark.py
# 4 cases, all PASS
```

```
# Layer 4 -- End-to-end cosim (~5 minutes)
cd boards/tangnano9k/neorv32_mmio/cosim
make clean
make SIMULATOR=verilator
# UART output streams to stdout and to neorv32_ascon_tb.log

# Standalone silicon evidence
make -C boards/tangnano9k/ascon_aead128_kat_slow bitstream
sudo make -C boards/tangnano9k/ascon_aead128_kat_slow prog-sram
# PASS LEDs

make -C boards/tangnano9k/ascon_aead128_mmio_slow bitstream
sudo make -C boards/tangnano9k/ascon_aead128_mmio_slow prog-sram
# LED1+LED2+LED4+LED5 on, LED3 off

# Integrated SoC (synth OK, PnR fails on GW1NR-9)
make -C boards/tangnano9k/neorv32_mmio bitstream
# utilisation table matches §6.2 above; nextpnr halts at utilisation limit
```

C Glossary

Term	Definition
ABI	Application Binary Interface — in this project, the firmware-facing register protocol of the accelerator.
AEAD	Authenticated Encryption with Associated Data.
AXI-stream	ARM AMBA streaming bus protocol used as the data-plane interface for the higher-throughput backends.
CFS	Custom Functions Subsystem — NEORV32's MMIO accelerator slot.
Cosim	Co-simulation: running the firmware ELF on a cycle-accurate model of the CPU driving the real RTL.
GHDL	GPL-licenced VHDL simulator and synthesiser, used here as a VHDL→Verilog elaborator.
KAT	Known Answer Test, the bit-exact vectors used to verify cryptographic implementations.
LWC API	GMU Lightweight Cryptography Hardware API, the interface convention used by <code>ascon-hardware</code> .
LUT4	4-input Look-Up Table, the basic logic resource of the Gowin GW1NR-9 die.
MMIO	Memory-Mapped I/O.
NEORV32	Open-source RISC-V soft-core, used here as the host CPU for the CFS integration.
$p[n]$	The Ascon permutation, applied for n rounds.
RTL	Register-Transfer-Level hardware description.

D Document History

Rev.	Date	Notes
1.0	May 20, 2026	Initial development report, RASD-matched style, companion to RASD Rev. 0.5.